

Frontend Interview Handbook — Part 5: Performance, Core Web Vitals & Browser Internals

SECTION A: CORE WEB VITALS

Q1: Name and explain all three Core Web Vitals.

Answer:

- **LCP (Largest Contentful Paint):** Time until the largest visible content element (usually a hero image or heading) finishes rendering. Target: under 2.5s. Measures perceived *loading* speed.
- **CLS (Cumulative Layout Shift):** A score measuring how much visible content unexpectedly shifts position during the page's lifecycle. Target: under 0.1. Measures visual *stability*.
- **INP (Interaction to Next Paint):** Time from a user interaction (click, tap, key press) to the next visual update reflecting that interaction, measured across the page's full lifespan (not just one sample). Target: under 200ms. Measures *responsiveness*. (Replaced FID — First Input Delay — as the official metric in March 2024.)

Why asked: Baseline literacy check — if you can't name and define all three cleanly, it signals you haven't engaged with performance seriously, which is a red flag at a performance/SEO-focused company.

Q2: What specifically causes poor LCP, and how do you fix each cause?

Answer:

Cause	Fix
Slow server response (TTFB)	Better hosting/CDN, caching, SSG/ISR instead of per-request SSR where possible
Render-blocking CSS/JS	Inline critical CSS, defer non-critical JS, minimize third-party scripts
Slow-loading hero image	Compress/resize properly, serve modern formats (WebP/AVIF), use <code><link rel="preload"></code> for the LCP image, don't lazy-load it
Client-side rendering delay	Move to SSR/SSG so content is present without waiting on JS execution + data fetch

```
<link rel="preload" as="image" href="/hero.webp" />
```

Q3: What specifically causes poor CLS, and how do you fix each cause?

Answer:

Cause	Fix
Images/videos without dimensions	Always set <code>width</code> / <code>height</code> or <code>aspect-ratio</code>
Web fonts causing text reflow (FOUT)	<code>font-display: optional</code> or preload fonts with matching fallback metrics
Ads/embeds injected without reserved space	Reserve a fixed-size container up front
Content inserted above existing content dynamically	Avoid it; if unavoidable, animate smoothly rather than causing a jump

Q4: What specifically causes poor INP, and how do you fix each cause?

Answer:

Cause	Fix
Long JS tasks blocking the main thread	Break up long tasks, use <code>requestIdleCallback</code> /code-splitting
Heavy synchronous work in event handlers	Debounce/throttle, defer non-critical work
Large hydration cost (whole app hydrating at once)	Use RSC/partial hydration so less JS needs to hydrate before interactivity
Excessive re-renders from state changes	Proper memoization, avoid unnecessary context re-renders

Follow-up: "How would you measure INP in production, not just locally?" A: Real User Monitoring (RUM) via `web-vitals` JS library reporting to an analytics endpoint, or Chrome UX Report (CrUX) data in Search Console/PageSpeed Insights, since lab tools (Lighthouse) don't capture real user interaction variance.

Q5: What tools do you use to measure and debug Core Web Vitals?

Answer: **Lighthouse** (lab data, in Chrome DevTools or CLI) for a controlled synthetic audit. **PageSpeed Insights** — combines Lighthouse lab data with real CrUX field data. **Chrome DevTools Performance panel** for deep profiling of what's actually happening on the main thread. **web-vitals** npm package for capturing real user metrics and sending them to your own analytics. **Search Console's Core Web Vitals report** for aggregated real-world data across a whole site over time.

```
js

import { onLCP, onCLS, onINP } from 'web-vitals';
onLCP(console.log);
onCLS(console.log);
onINP(console.log);
```

SECTION B: PERFORMANCE OPTIMIZATION

Q6: What is code splitting, and how do you implement it in React/Next.js?

Answer: Code splitting breaks a single large JS bundle into smaller chunks loaded on demand, so users only download the code needed for the current view instead of the entire app upfront. React supports this via `React.lazy` + `Suspense`; Next.js does automatic route-based code splitting out of the box, plus manual splitting via `next/dynamic`.

```
jsx

const HeavyChart = React.lazy(() => import('./HeavyChart'));

function Dashboard() {
  return (
    <Suspense fallback={<Spinner />}>
      <HeavyChart />
    </Suspense>
  );
}
```

Q7: What's the difference between lazy loading and preloading/prefetching, and when do you use each?

Answer: **Lazy loading** defers loading a resource until it's actually needed (e.g., an image below the fold, or a route the user hasn't navigated to yet) — saves bandwidth/initial load time. **Preloading** tells the browser to fetch a resource early because it will definitely be needed soon (e.g., the LCP hero image, a font). **Prefetching** speculatively fetches a resource

that will *likely* be needed next (e.g., `next/link`) prefetching the JS for a linked page on hover/viewport-entry) — lower priority than preload, used for probable-but-not-certain next steps.

Q8: How do you optimize images for the web in a real production app?

Answer: Serve appropriately-sized images per viewport (`srcset`)/responsive images, or `next/image` handling it automatically), use modern compressed formats (WebP/AVIF with fallback), lazy-load offscreen images but *not* the LCP image, compress without visible quality loss, and serve from a CDN with proper caching headers. For a real estate or product site (relevant to your Rentmann experience), this alone can be one of the highest-leverage performance fixes since images are often the largest payload on the page.

Q9: What's the difference between bundling, minification, and tree-shaking?

Answer: **Bundling** combines multiple JS/CSS modules into fewer files to reduce request overhead. **Minification** strips whitespace/comments and shortens variable names to reduce file size without changing behavior. **Tree-shaking** removes code that's never actually used/imported (dead code elimination) — relies on ES module static structure (`import`/`export`) to determine what's safe to drop, which is part of why CommonJS (`require`) is harder to tree-shake effectively.

Q10: How would you diagnose a real production page that's "slow" with no other information?

Answer: Structured approach to walk through out loud:

1. Run Lighthouse/PageSpeed Insights first for a quick overview of which specific metric (LCP/CLS/INP) is failing.
2. Check the Network tab — look for render-blocking resources, oversized payloads, waterfall gaps (sequential requests that could be parallelized).
3. Check the Performance tab for long main-thread tasks if INP/interactivity is the complaint.
4. Check whether the page is CSR-heavy (blank initial HTML) vs SSR/SSG.
5. Check third-party scripts (analytics, ads, chat widgets) — often the actual culprit, frequently overlooked because it's "not our code."

Why this question matters: Interviewers care more about your diagnostic process than a memorized answer — narrate a clear, ordered approach rather than guessing randomly.

SECTION C: BROWSER INTERNALS

Q11: Walk through what happens between typing a URL and seeing the page rendered.

Answer (concise version to say out loud): DNS lookup resolves the domain to an IP → TCP connection (and TLS handshake for HTTPS) established → browser sends HTTP request → server responds with HTML → browser parses HTML into the DOM, discovering and requesting CSS/JS/images along the way → CSS parsed into CSSOM → DOM + CSSOM combine into the render tree → **layout** (compute size/position of every element) → **paint** (draw pixels onto layers) → **composite** (layers combined onto the screen). If any render-blocking JS is hit during HTML parsing, parsing pauses until it downloads and executes.

Q12: What's the difference between the main thread and Web Workers, and when would you use a worker?

Answer: JS is single-threaded on the main thread by default — this thread also handles layout, paint, and user interaction, so a long-running JS task blocks all of that (visible as a frozen/unresponsive page). Web Workers run JS in a separate background thread with no DOM access, communicating with the main thread via message passing — useful for CPU-heavy work (large data processing, image manipulation, complex calculations) that would otherwise jank the UI.

Q13: What's the difference between `localStorage`, `sessionStorage`, and cookies?

Answer: `localStorage` persists indefinitely (until explicitly cleared), scoped per origin, not sent to the server automatically. `sessionStorage` persists only for the tab's lifetime, same origin scoping. Cookies are small (~4KB), can have expiration, are automatically sent with every matching HTTP request (relevant for auth), and can be scoped with `httpOnly` (inaccessible to JS, safer against XSS) and `Secure`/`SameSite` flags. For auth tokens specifically, `httpOnly` cookies are generally safer than `localStorage`, which is readable by any JS running on the page (including injected via XSS).

Q14: What is repaint vs reflow (layout), and why is reflow more expensive?

Answer: Reflow/layout recalculates the geometry (size/position) of elements — triggered by things like changing `width`, adding/removing DOM nodes, or reading certain layout properties (`offsetHeight`) that force synchronous recalculation. **Repaint** just redraws pixels without recalculating geometry — triggered by changes like `color` or `background-color` that don't affect layout. Reflow is more expensive because it can cascade — changing one element's size can force recalculation of its siblings/ancestors/descendants. Animating `transform`/`opacity` instead of `width`/`top`/`left` avoids both reflow and often even repaint, using only cheap compositing — this is why they're recommended for smooth animations.

Rapid-Fire Round

- **Q: What's a CDN and why does it help performance?** A: Content Delivery Network — serves cached content from geographically distributed edge servers close to the user, reducing latency versus a single origin server.
- **Q: What's HTTP/2 vs HTTP/1.1 relevant performance difference?** A: HTTP/2 multiplexes multiple requests over a single connection, removing the old "6 connections per domain" bottleneck that led to bundling/sprite hacks.
- **Q: What's critical CSS?** A: The minimal CSS needed to render above-the-fold content, inlined directly in `<head>` so the browser doesn't have to wait on an external stylesheet request before first paint.
- **Q: What's a service worker?** A: A script that runs separately from the page, enabling offline support, caching strategies, and push notifications by intercepting network requests.
- **Q: Why avoid too many third-party scripts?** A: Each one adds network requests, main-thread execution time, and often blocks rendering — a very common real-world cause of poor Core Web Vitals despite "your own code" being fine.

End of Part 5. Say "next part" for Part 6: APIs, React Query, State Management & Security.